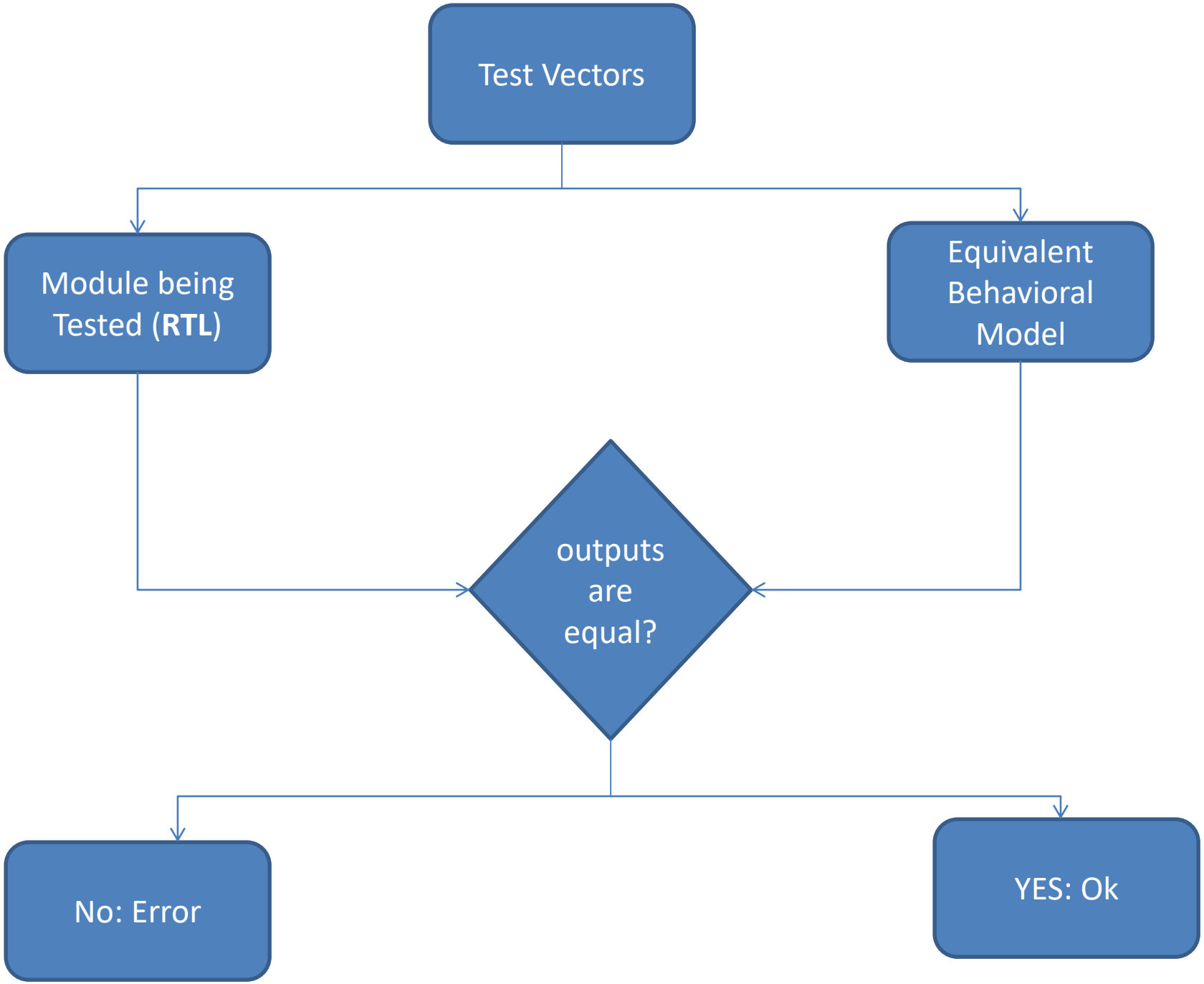
Testbench in VERILOG

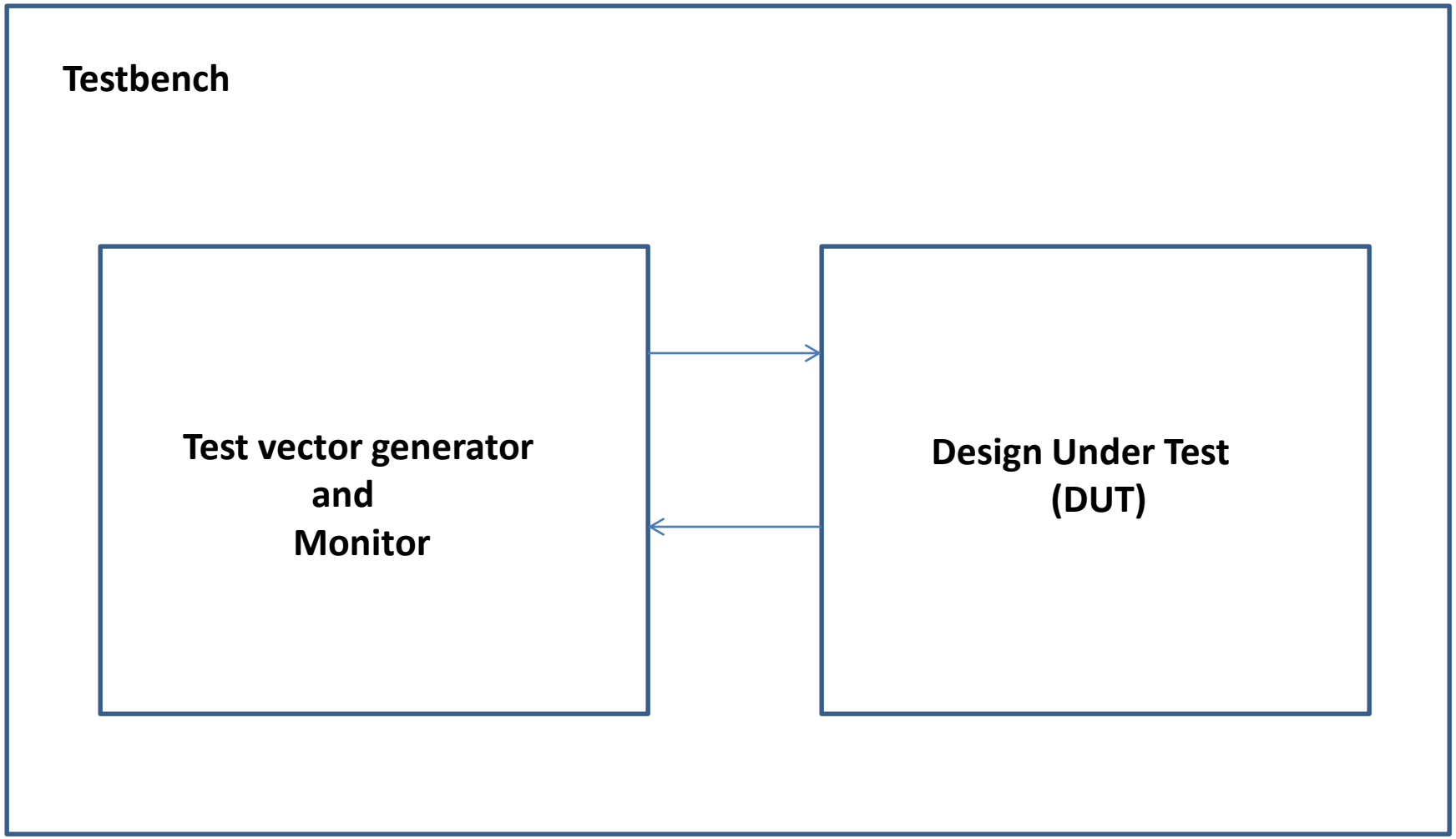
Dr. Chandan Karfa

Department of Computer Science and Engineering

भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

NPTEL


Test Vectors
Module being Tested (**RTL**)
Equivalent Behavioral Model
outputs are equal?
No: Error
YES: Ok

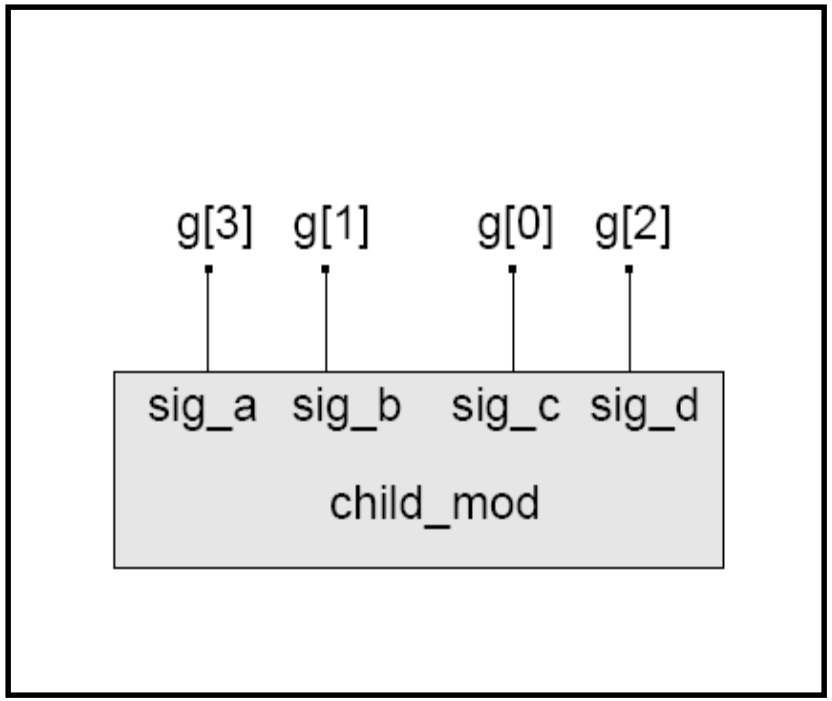

Testbench
Test vector generator
and
Monitor
Design Under Test
(DUT)

Connection by Position

```
module child_mod (sig_a, sig_b,
sig_c, sig_d);
  input      sig_a, sig_b;
  output     sig_c, sig_d;

  // module description goes here.

endmodule
```

```
module parent_mod;
   wire [4:0] g;

   child_mod G1 (g[3], g[1], g[0], g[2]);

   // Listed order is significant
endmodule
```

parent_mod

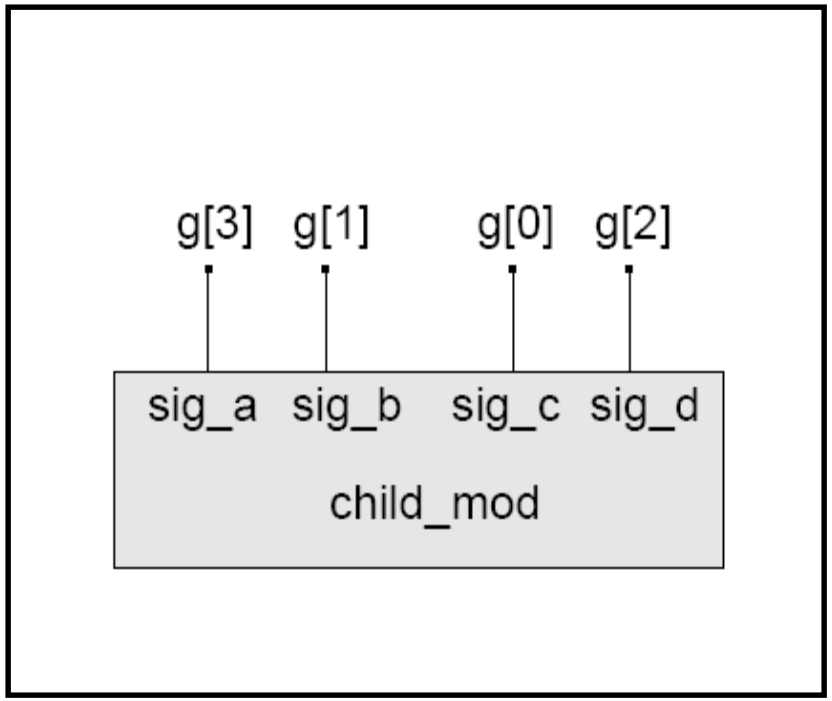


Connection by Name

```
module child_mod (sig_a, sig_b,
sig_c, sig_d);
  input      sig_a, sig_b;
  output     sig_c, sig_d;

  // module description goes here.

endmodule
```

```
module parent_mod;
    wire [3:0] g;
   //order is not significant
    child_mod G1 ( .sig_c(g[0]),
                   .sig_a(g[3]),
                   .sig_d(g[2]),
                   .sig_b(g[1]));
endmodule
```

parent_mod



Recommended style!

Delay

Time duration between assignment from RHS to LHS.

- Inter assignment delay (**Delayed execution**)

  #10 q = x + y ;

  It simply waits for the appropriate number of time steps before executing the command.

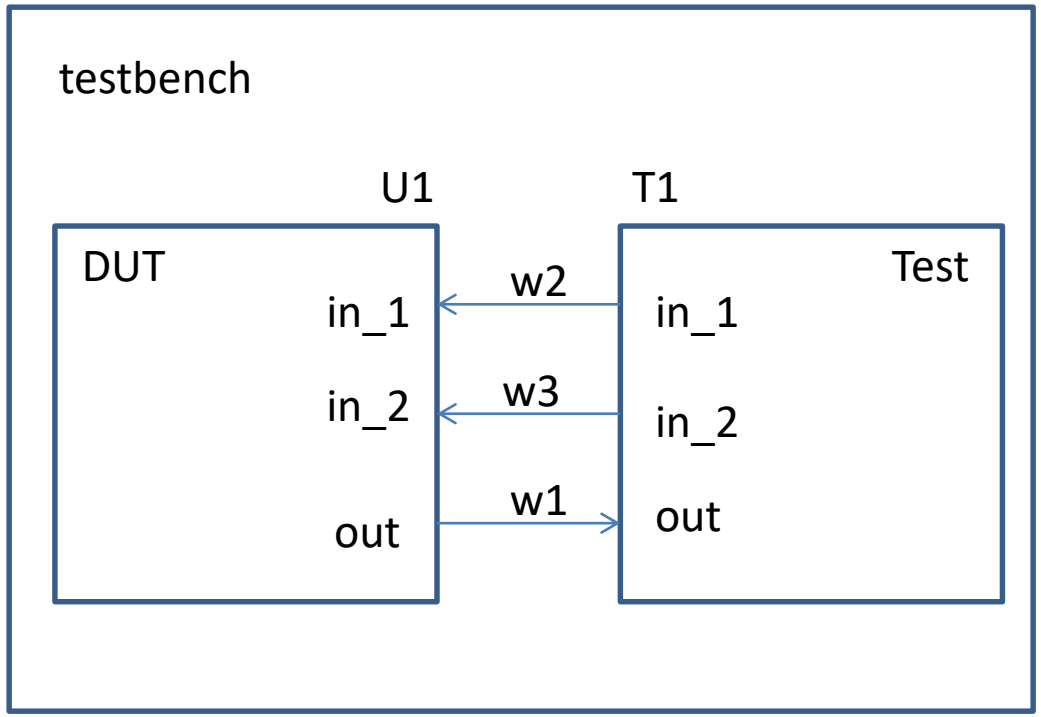
- Intra assignment delay (**Delayed assignment**)

  q = #10 x + y ;

  Value of x + y is calculated at the time that the assignment is executed, but this value is not assigned to q until after the delay period, regardless of whether or not x or y have changed during that time.

Inter-assignment	**#5 a = b ;**	**#5 ;** **a = b ;**
Intra-assignment	a = #5 b ;	temp = b ; #5 ; a = temp ;

```
module testbench ;
  wire w1, w2, w3 ;
  DUT U1 (w1, w2, w3) ;
  Test  T1  (w1, w2, w3) ;
endmodule


module DUT (out, in_1, in_2) ;
   output out ;
   input in_1, in_2 ;
   assign out = in_1 & in_2 ;
endmodule


module Test (out, in_1, in_2) ;
  input out ;
  output reg in_1, in_2 ;
   initial
   begin
     $monitor ($time, ": in_1 = %b, in_2 =
%b, out = %b", in_1, in_2, out) ;
          in_1 = 0 ; in_2 = 0 ;
     #10 in_1 = 0 ; in_2 = 1 ;
     #10 in_1 = 1 ; in_2 = 0 ;
     #10 in_1 = 1 ; in_2 = 1 ;
     #10 $finish ;
   end
endmodule
```



Output Log:

0: in_1 = 0, in_2 = 0, out = 0
10: in_1 = 0, in_2 = 1, out = 0
20: in_1 = 1, in_2 = 0, out = 0
30: in_1 = 1, in_2 = 1, out = 1

Compiler Directives and System Tasks

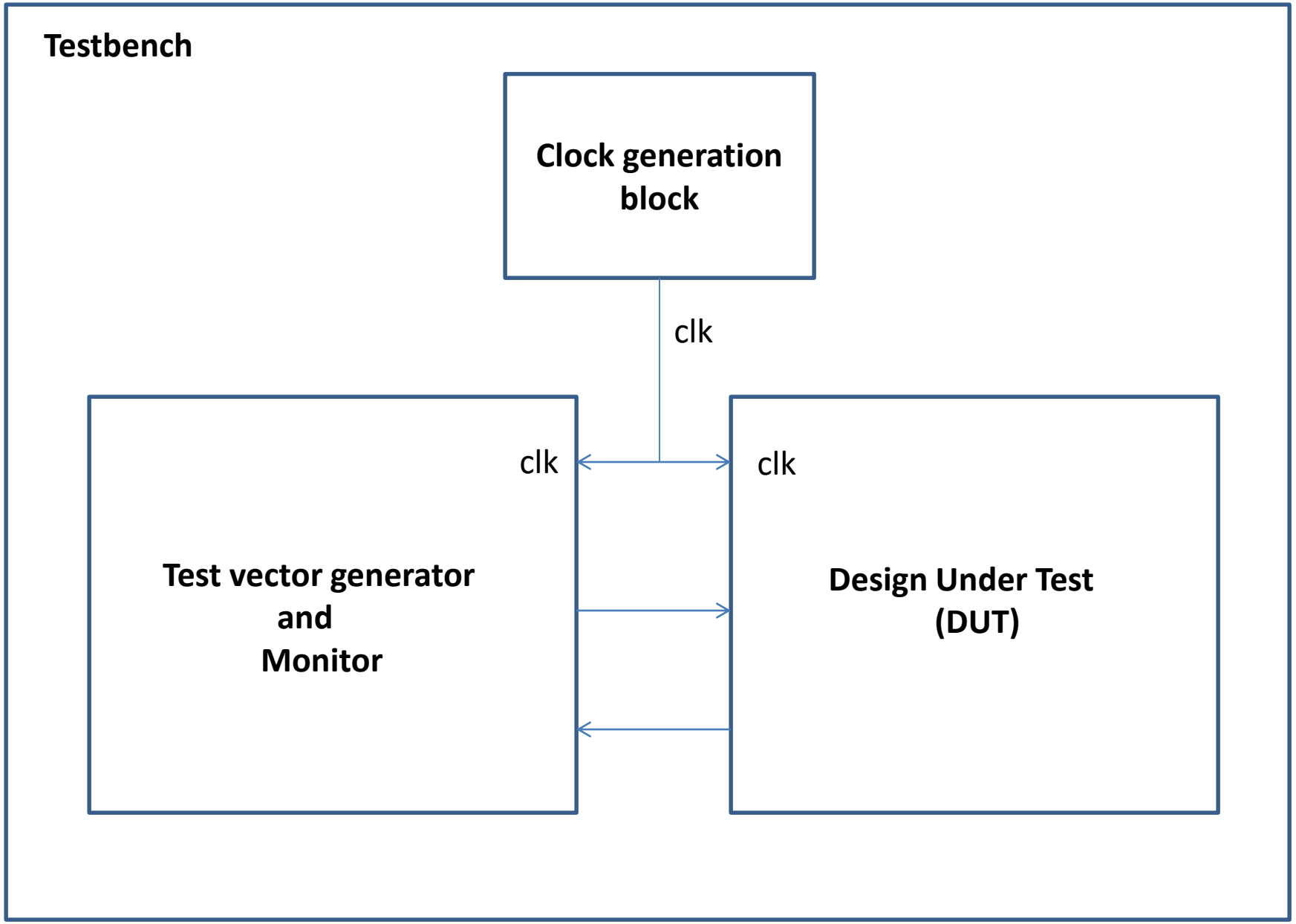
- **`define**: used to define a text macro.
- **`include**: used to include content of some other file inside a verilog code.
- **$time**: It returns the current simulation time as 64-bit integer value. However this value will be scaled to the `timescale unit.
- **$monitor**: It continuously monitors the changes in any one of the variable specified in the parameter list. Whenever, anyone of them changes, it displays the formatted string specified within double qoutes.
- **$finish** : It terminates the simulation.

Compiler Directives and System Tasks

`timescale <ref_time_unit>/<precision>
Example: `timescale 1ns/ 100 ps
Time values to be read as **ns** and to be rounded to the nearest 100 **ps**.

`timescale 10ns / 10ns
#1.5 ;
1.5 x 10 ns = 15 ns.
Rounded to 20 ns.

`timescale 1ns / 1ps
#1 ; // 1 ns delay
#0.001; // 1 ps. This is the minimum delay you can have with this time scale.
#0.0001; // this will give 0 ns delay.


Testbench
Clock generation block
clk
clk
clk
Test vector generator and Monitor
Design Under Test (DUT)

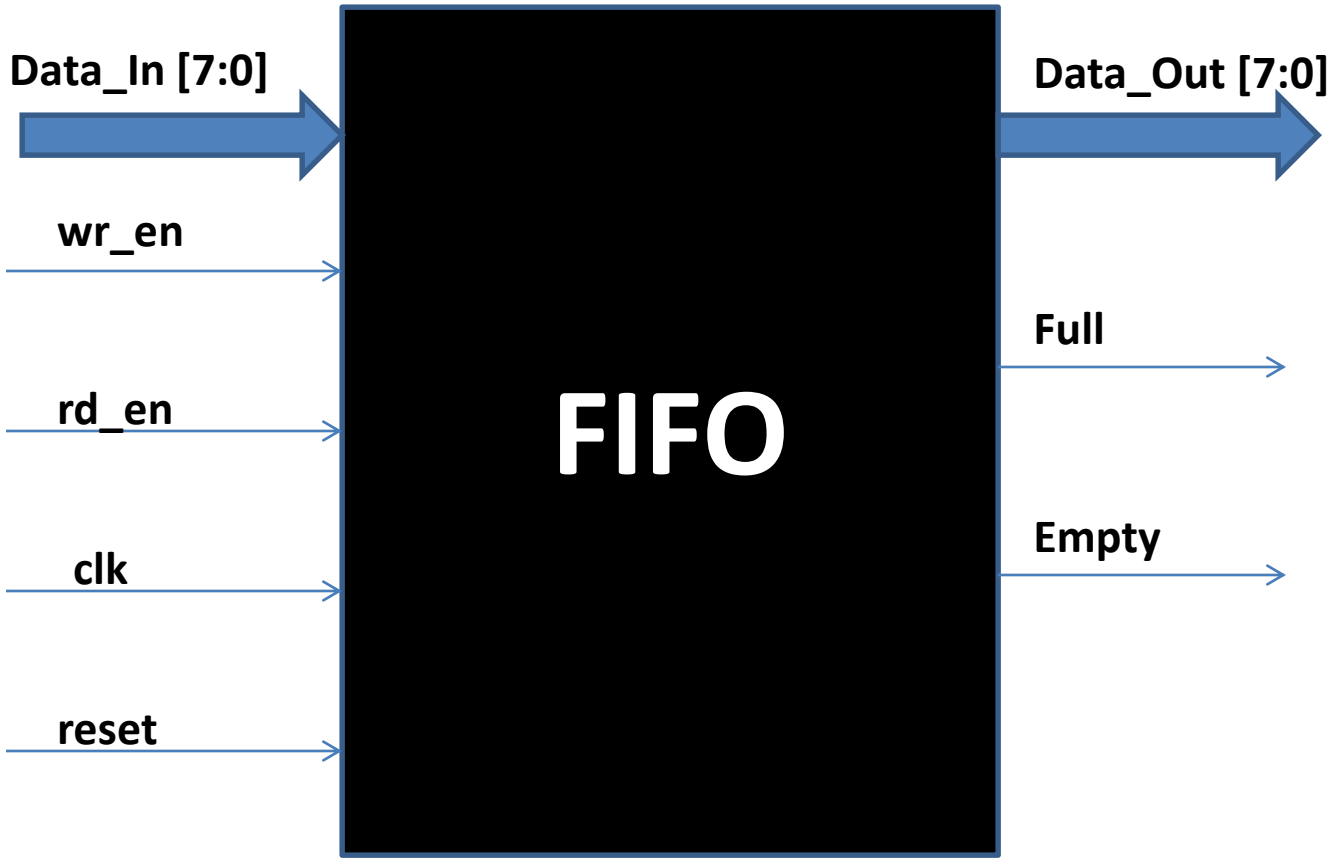
Clock generation with 50% duty cycle

```
initial clk = 0 ;
always #10 clk = ~clk ;

always
begin
   clk = 0 ;
   #10 ;
   clk = 1 ;
   #10 ;
end

parameter CLOCK_PERIOD = 5
always # (CLOCK_PERIOD / 2) clk = ~clk ;
```

Clock generation with variable duty cycle

```
module clk_gen (output clk) ;
  parameter CLK_PERIOD = 10;
  parameter DUTY_CYCLE = 60; //60% duty cycle
  parameter TCLK_HI = (CLK_PERIOD*DUTY_CYCLE/100);
  parameter TCLK_LO = (CLK_PERIOD-TCLK_HI);

  reg clk;

  initial
  clk = 0;

  always
  begin
    #TCLK_LO;
    clk = 1'b1;
    #TCLK_HI;
    clk = 1'b0;
  end
endmodule
```

Example: Synchronous FIFO



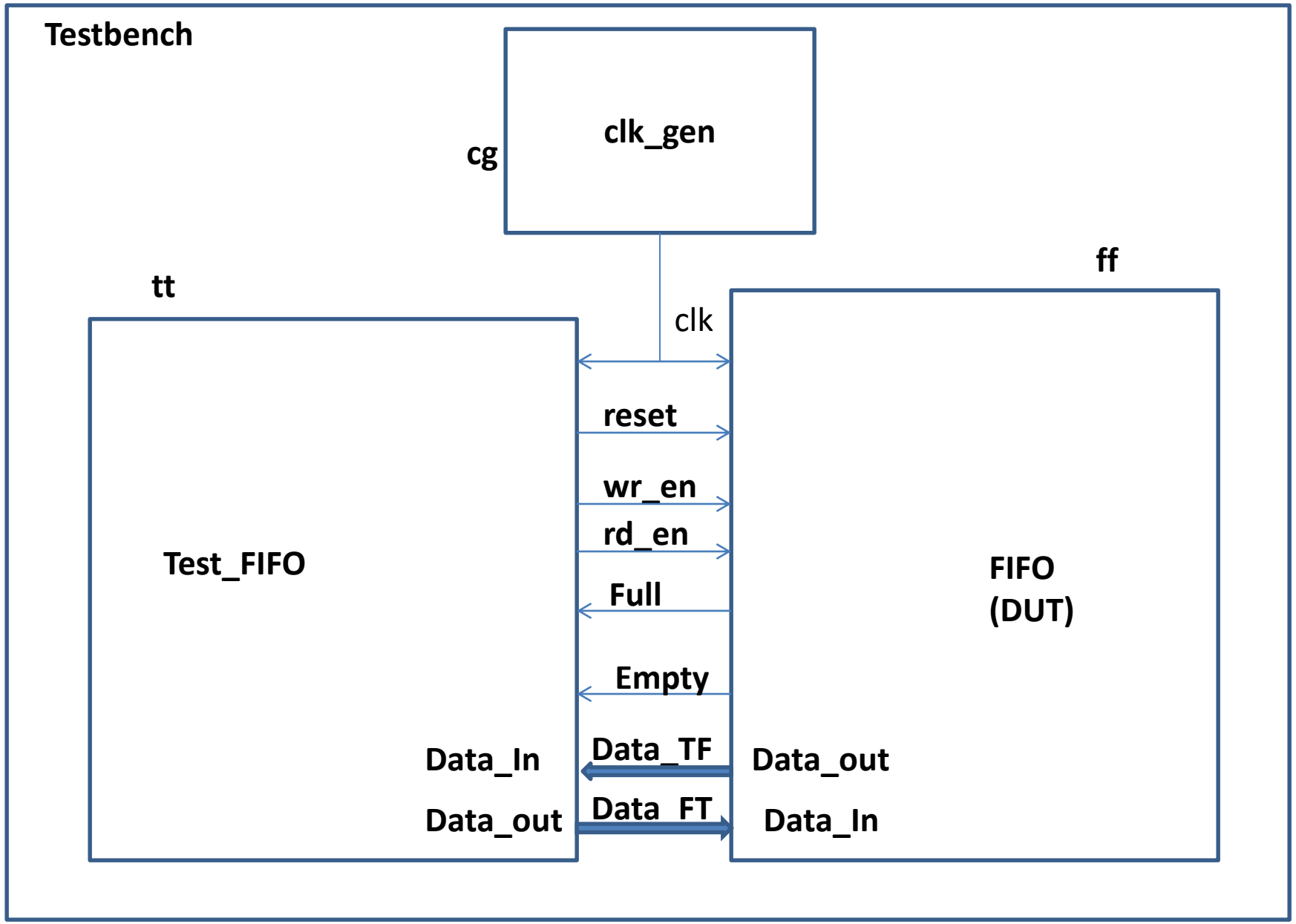

```
module FIFO # (parameter DATA_WIDTH = 8,
                parameter FIFO_DEPTH = 10)
               (Data_out, Full, Empty, Data_In, Wr_En_
Rd_En, clk, reset) ;
```

```
module FIFO # (parameter DATA_WIDTH = 8,
                parameter FIFO_DEPTH = 32)
               (Data_out, Full, Empty, Data_In, wr_en_ rd_En, clk, reset) ;

    input [DATA_WIDTH – 1 : 0] Data_In ;
    input wr_en ;
    input rd_en ;
    input clk ;
    input reset ;

    output reg [DATA_WIDTH – 1 : 0] Data_Out ;
    output reg Full ;
    output reg Empty ;

    reg [DATA_WIDTH – 1 : 0] memory [FIFO_DEPTH - 1: 0] ;
    reg [FIFO_DEPTH – 1 : 0] rd_ptr, wr_ptr, depth_cnt ;
```

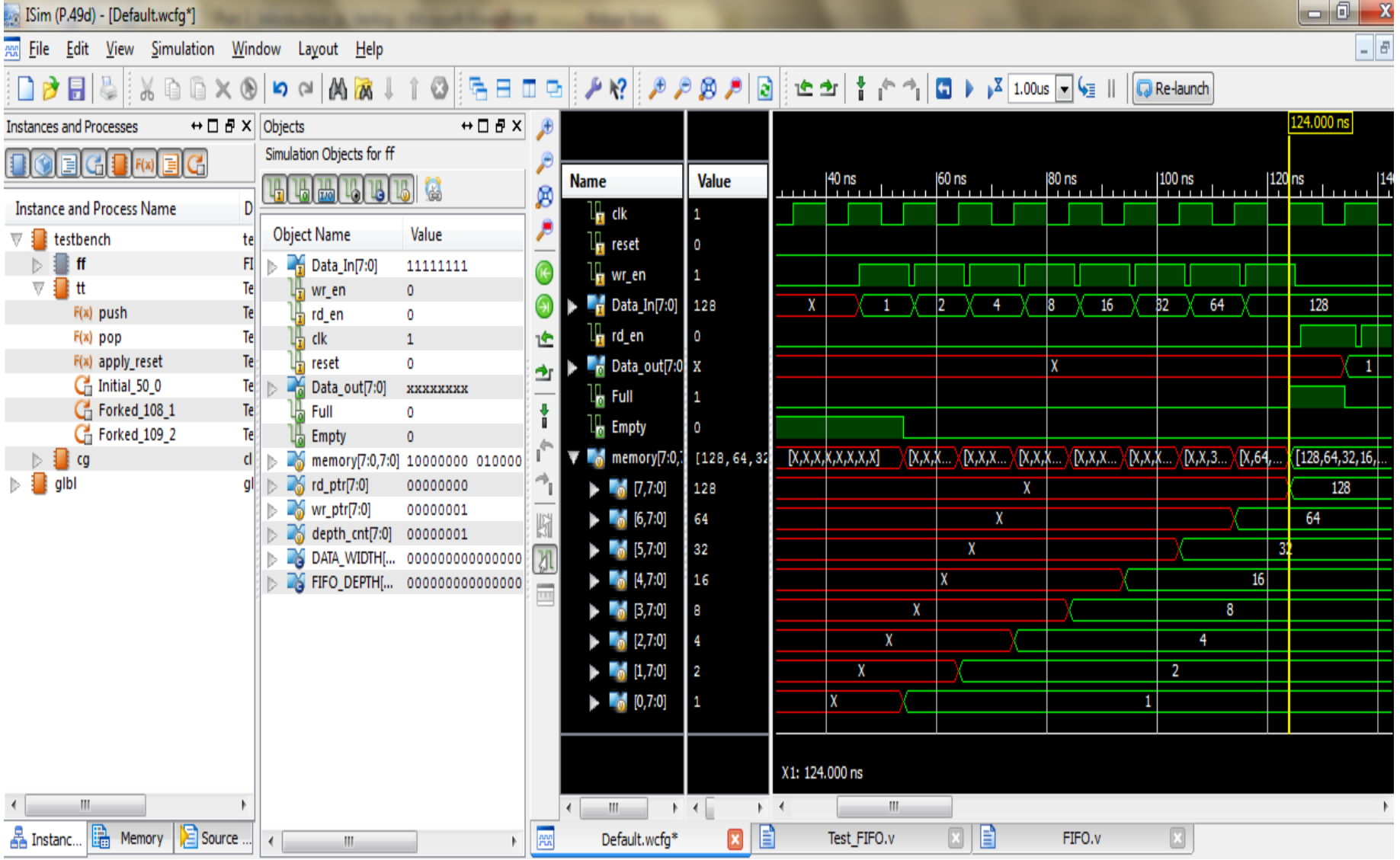
```
//PUSH
always @ (posedge clk)
begin
  if (reset) begin
    wr_ptr <= 'h0 ;
  end
  else begin
    if (wr_en && !Full) begin
      memory [wr_ptr] <= Data_In ;
      wr_ptr <= wr_ptr + 1 ;
    end
  end
end
```

```
//POP
always @ (posedge clk)
begin
  if (reset) begin
    rd_ptr <= ‘h0 ;
  end
  else begin
    if (rd_en && !Empty) begin
      Data_out <= memory [rd_ptr] ;
      rd_ptr <= rd_ptr + 1 ;
    end
  end
end
```

```
//Depth Count
always @ (posedge clk)
begin
  if (reset) begin
    depth_cnt <= 'h0 ;
  end
  else begin
    if (wr_en && !Full)
      depth_cnt <= depth_cnt + 1 ;
   else if (rd_en && !Empty)
      depth_cnt <= depth_cnt - 1 ;
    //add one more condition to check simultaneous WR & RD.
  end
end

  assign Empty = (depth_cnt == 'h0)? 1 : 0 ;
  assign Full     = (depth_cnt == FIFO_DEPTH)? 1 : 0 ;
endmodule
```


Testbench
clk_gen
cg
tt
ff
clk
reset
wr_en
rd_en
Full
Empty
Test_FIFO
FIFO
(DUT)
Data_In
Data_out
Data_TF
Data_FT
Data_out
Data_In

```
module testbench # (parameter DATA_WIDTH = 8,
                    parameter FIFO_DEPTH = 8);
  wire clk, reset ;
  wire rd_en, wr_en, Full, Empty ;
  wire [DATA_WIDTH - 1 : 0] data_tf, data_ft;

 FIFO ff(.clk (clk), .reset (reset),
            .Full (Full), .Empty (Empty),
            .wr_en (wr_en), .rd_en(rd_en),
            .Data_In (data_tf), .Data_out (data_ft)) ;

 Test_FIFO  tt(.Data_out (data_tf), .Full (Full),
              .Empty (Empty), .Data_in(data_ft),
              .wr_en(wr_en), .rd_en(rd_en),
              .clk (clk), .reset(reset)) ;

  clk_gen cg(.clk (clk)) ;

endmodule
```



From Instance and Process Name, select “design Top”. Then from “Objects”, select required objects and the right click to “add to wavewindow”.

Verilog Coding Guidelines

- When modeling sequential logic, use non-blocking assignments.
- When modeling latches, use non-blocking assignments.
- When modeling combinational logic with an always block, use blocking assignments.
- When modeling both sequential and combinational logic within the same always block, use non-blocking assignments.
- Do not mix blocking and non-blocking assignments in the same always block.
- Do not make assignments to the same variable from more than one always block.

Thank You